ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -



CET214

Web Programming

02 - HTML5 Fundamentals

Dr. Ahmed Said

© 2026

Learning Objectives

- Understand the purpose and structure of HTML
- Write a valid HTML5 document from scratch (`lang`, `charset`, `viewport`)
- Use common HTML elements: headings, paragraphs, links, images, and lists
- Serve responsive, fast-loading images (`srcset`, `<picture>`, `loading="lazy"`)
- Create HTML tables with proper semantic structure
- Distinguish between block-level and inline elements
- Apply character entities for special characters

What is HTML?

- **H**yper**T**ext **M**arkup **L**anguage
- Describes the **content** and **structure** of information on a web page
- Not the same as **presentation** (appearance) - that is CSS!
- HTML uses **tags** to annotate content so the browser knows what each piece is
- The current standard is the **HTML Living Standard** (WHATWG) - everyone still calls it **HTML5**

> `Key Resource`
>
> **MDN Web Docs** is the best reference for HTML: developer.mozilla.org/en-US/docs/Web/HTML

HTML Page Structure

Goal: write a complete, valid HTML document skeleton and explain every line.

Basic Structure of an HTML Page

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

Shortcut

In VS Code, type `!` then press `Tab` - Emmet writes this skeleton for you. Tags are **case-insensitive**, but the convention is **lowercase**.

- `<!DOCTYPE html>` - use **standards mode** (not legacy "quirks mode")
- `<html lang="en">` - the root element; `lang` tells screen readers & translators the language
- `<meta charset="UTF-8">` - text encoding; put it **first** in `<head>`
- `viewport` - makes the page fit phone screens instead of a zoomed-out desktop view
- `<title>` - required; shown in the tab and in search results
- `<body>` - all the **visible content** of the page

HTML Tag Attributes

Some tags can contain additional information called **attributes**

```html
<element attribute="value" attribute="value">content</element>
```

- Attributes provide extra information about an element
- They are always specified in the **opening tag**, as `name="value"` pairs (quote the values)
- Common attributes: `href`, `src`, `alt`, `class`, `id`, `lang`
- **Boolean attributes** need no value: `<input required>`, `<video controls>`

Examples:

```html
<a href="about.html">About Us</a>
<img src="logo.png" alt="Company Logo">
<p class="intro" id="main-intro">Welcome!</p>
```

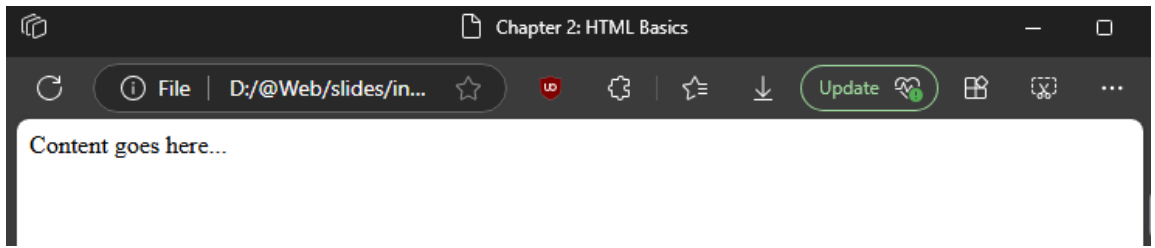
Void Elements

- Some elements have no content and no closing tag
- **void elements**: `<br>`, `<hr>`, `<img>`, `<input>`, `<meta>`, `<link>`.
- The trailing `/` (`<br />`) is an XHTML habit: allowed but ignored in HTML this course writes `<br>`.

☆ Page Title & Favicon

Page Title: displayed in the browser tab and bookmarks:

```html
<head>
  <title>Chapter 2: HTML Basics</title>
</head>
```



Favicon: a small icon associated with the website:

```html
<link rel="icon" href="favicon.svg" type="image/svg+xml">
```

- The `<title>` is placed inside `<head>` - it does **not** appear on the page itself
- Make titles **unique and descriptive**: "Contact - Cairo Bakery", not "Page 1"
- Favicons appear in browser tabs, bookmarks, and history - `rel="icon"` (the old `rel="shortcut icon"` is obsolete)
- More favicon formats and sizes in the **Semantics & Metadata** lecture

T Text Elements

Goal: mark up text, links and images so they are meaningful, accessible and fast.

P Paragraphs & Headings

Paragraph: `<p>` defines a block of text:

```html
<p>This is a paragraph of text.</p>
```

Headings: six levels from `<h1>` (largest) to `<h6>` (smallest):

```html
<h1>Main Title</h1>
<h2>Section Title</h2>
<h3>Subsection</h3>
```

Main Title

Section Title

Subsection

- Both are **block-level** elements
- Browsers add spacing above and below automatically
- Use headings to create a **logical document outline** - do not skip levels
- Choose a heading by its **level in the outline**, not its size - size is CSS's job

Links (Anchors): `<a>`

Links allow navigation between pages using the `href` (Hypertext REFerence) attribute.

```html
<p>Search for it on <a href="https://www.google.com/">Google</a>!</p>
<p>Read the <a href="#rules">course rules</a> below.</p>
<p>Email <a href="mailto:me@example.com">the instructor</a>.</p>
```

Search for it on Google!

Read the course rules below.

Email the instructor.

- `href` can point to another site, a **fragment** on the same page (`#id`), or a scheme like `mailto:` / `tel:`
- Anchors are usually used **inline** inside text - but in HTML5 an `<a>` may also wrap a whole card of block content
- Write link text that makes sense out of context: "course rules", not "click here"

Relative vs. Absolute Paths

- **Relative paths** - relative to the current document's location:
  - Same directory: `"about.html"`
  - Subdirectory: `"img/photo.jpg"`
  - Parent directory: `"../index.html"`
  - Site root: `"/img/photo.jpg"` - careful: on GitHub Pages project sites the root is `username.github.io`, not your repo
- **Absolute paths** - include the full URL with protocol and domain:
  - `"https://www.google.com/some-image.png"`

```html
<!-- Relative -->
<a href="contact.html">Contact Us</a>
<img src="images/logo.png" alt="Logo">
<!-- Absolute -->
<a href="https://www.google.com/">Google</a>
```

Best Practice

Use **relative paths** for files within your own site and **absolute paths** for external resources. Paths on web servers are **case-sensitive**: `Logo.png` ≠ `logo.png`.

Images: `<img>`

Inserts a graphical image into the page (inline element).

```html
<img src="img/html5.jpg"
     alt="HTML5 Logo"
     width="400" height="400">
```

- `src` - specifies the image URL
- `alt` - describes the image for **accessibility**
- `width` / `height` - intrinsic size in pixels (no `px`); lets the browser **reserve space** before the image loads
- If inside an `<a>` tag, the image becomes a **clickable link** - its `alt` becomes the link text
- Avoid relying on `title` tooltips: touch and keyboard users never see them




```html
<!-- Image as a link -->
<a href="https://example.com">
  <img src="logo.png" alt="Example.com home">
</a>
```

Why `alt` Text Matters

- Improves **accessibility** for users with screen readers
- Displayed when the image **fails to load**
- Helps **search engines** understand image content (SEO)
- **Required** attribute - for purely decorative images use an **empty** `alt=""` so screen readers skip them
- Describe the purpose, not the file: `alt="Sales rose 15% in Q3"`, not `alt="chart.png"`

Image loaded:



Image broken - alt text shown:

HTML5 Logo

Quick Quiz: Links, Paths & Images

Q1. Your page is `site/blog/post.html`. Write the `src` for `site/img/cat.jpg`.

✅ `../img/cat.jpg` - go up one folder, then into `img`.

Q2. What `alt` should a purely decorative divider image have?

✅ `alt=""` (empty, but present) - screen readers skip it.

Q3. Why add `width` and `height` to the image tag?

✅ They give the browser the **aspect ratio**, so it reserves space and the text below does not jump.

HTML Lists

Goal: choose the right list type - ordered, unordered, description or nested.

Ordered Lists

Numbered lists use `<ol>` with `<li>` items. The browser numbers them automatically.

```html
<ol>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
```

1. First item
2. Second item
3. Third item

```html
<ol start="5" reversed>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
```

5. First item
4. Second item
3. Third item

Customization

- Want letters or Roman numerals? Use the **CSS** `list-style-type` property.
- Need to count from 5 or backwards? Use the HTML attributes `start="5"` and `reversed`.

Unordered Lists

Bulleted lists use `<ul>` with `<li>` items.

```html
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

- HTML
- CSS
- JavaScript

- **Unordered lists** - order does not matter (ingredients, features, **navigation menus**)
- **Ordered lists** - order matters (steps, rankings)
- `<ul>` / `<ol>` may contain **only** `<li>` children
- Bullet style can be changed with **CSS**

📖 Definition Lists

A list of terms and their descriptions using `<dl>`, `<dt>`, and `<dd>`.

```html
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
  <dt>JS</dt>
  <dd>JavaScript</dd>
</dl>
```

HTML
: HyperText Markup Language

CSS
: Cascading Style Sheets

JS
: JavaScript

- `<dl>` - description list container
- `<dt>` - the term or name
- `<dd>` - the description for the preceding term
- Use it for glossaries, FAQs, and key–value metadata (e.g. "Author: …")

Nested Lists

A list can contain other lists - useful for hierarchical data.

```html
<ul>
  <li>Web Languages
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Server Languages
    <ul>
      <li>PHP</li>
      <li>Python</li>
    </ul>
  </li>
</ul>
```

- Web Languages
  - HTML
  - CSS
  - JavaScript
- Server Languages
  - PHP
  - Python

The nested `<ul>` goes **inside** the parent `<li>`, before its `</li>` - never directly inside the outer `<ul>`.

Try It Yourself: Profile Card

In VS Code, type `!` + `Tab`, then build this page **without any CSS** (8 minutes):

1. An `<h1>` with your name and a `<p>` introduction
2. Your photo (or any image) with good `alt`, `width` and `height`
3. An `<h2>My Skills</h2>` followed by an **unordered** list
4. An `<h2>My Week</h2>` followed by an **ordered** list of 3 steps
5. A link to your GitHub profile and a `mailto:` link
6. Open it with **Live Server** and inspect it in **DevTools → Elements**

✅ Self-check: exactly one `<h1>`, no skipped heading levels, every `<img>` has `alt`, every `<li>` is inside a list.

<> Character Entities, Element Types & Nesting

Goal: escape reserved characters, predict how block and inline elements lay out, and nest them legally.

!?# HTML Character Entities

A way of representing special or reserved HTML characters like `<`, `>`, and `&` within a web page.

Character(s)	Entity
< >	`<` `>`
" &	`"` `&`
© ™	`©` `™`
é ñ	`é` `ñ`
π Δ	`π` `Δ`
(non-breaking space)	` `

- Entities start with `&` and end with `;`
- Example: To display `<p>` as text, write `<p>`
- With `<meta charset="UTF-8">` you can type ©, é, π directly - you **must** escape only `<` and `&` (and `"` inside attribute values)

Challenge

How would you display the text `&` on a web page?

✅ Escape the ampersand itself: write `&` in the source.

Block vs. Inline Elements

Block Elements ↓

- Occupy the **full width** available
- Start on a **new line**
- Some have default margins (`<p>` , `<h1>` , `<ul>`); `<div>` has none
- Examples: `<p>` , `<h1>` - `<h6>` , `<div>` , `<ul>` , `<ol>` , `<table>` , `<blockquote>`

Inline Elements →

- Only take up **as much width as needed**
- Do **not** start on a new line
- Flow within surrounding text
- Examples: `<a>` , `<em>` , `<strong>` , `<img>` , `<code>` , `<span>`

Behind the scenes

"Block" and "inline" are really the default CSS `display` values - CSS can change them. The HTML spec itself talks about **flow content** and **phrasing content**.

Block vs. Inline: Example

```html
<em>text</em>
<em>text</em>
<em>text</em>

<p>text</p>
<p>text</p>
<p>text</p>
```

Output:

text text text

text

text

text

- The three `<em>` elements appear on the **same line** (inline)
- Each `<p>` element starts on a **new line** (block)
- `<p>` tags can only contain **inline** elements and plain text
- Note: the line breaks between the `<em>` tags in the source become **single spaces** in the output

ⓘ Nesting Rules

- **Block** elements can generally contain both block and inline elements
  - `<div>`, `<li>`, `<body>`, `<form>` - can contain either
- `<p>` tags can contain only **inline** elements and plain text - a `<div>` or `<ul>` inside `<p>` silently **closes** the paragraph
- Some elements are restricted in what they can contain:
  - `<ul>` / `<ol>` can only contain `<li>` (but `<li>` can contain nested lists!)
  - `<a>` must not contain another `<a>` or a `<button>`
- Only **once** per document: `<html>`, `<head>`, `<body>` - and only one visible `<main>`

> ⓘ **Rule of Thumb**
>
> When in doubt, use the **W3C Validator** to check if your nesting is correct: validator.w3.org

HTML Tables

Goal: build accessible data tables with headers, sections, captions and merged cells.

Basic Table Structure

Tables display data in rows and columns.

```html
<table>
  <tr>
    <th scope="col">Name</th>
    <th scope="col">Age</th>
  </tr>
  <tr>
    <td>Alice</td>
    <td>25</td>
  </tr>
  <tr>
    <td>Bob</td>
    <td>30</td>
  </tr>
</table>
```

Name	Age
Alice	25
Bob	30

- `<table>` - defines the table
- `<tr>` - defines a table **row**
- `<th>` - defines a **header** cell (bold, centered); `scope="col"` or `"row"` tells screen readers which cells it labels
- `<td>` - defines a **data** cell
- Tables are for **tabular data** only - never for page layout (that is CSS Grid/Flexbox)

Table Sections & Caption

Tables can be organized into **semantic sections** for better structure:

```
<!-- Table with semantic sections -->
<table>
  <caption>Fruit Prices</caption>
  <thead>
    <tr><th>Item</th><th>Price</th></tr>
  </thead>
  <tbody>
    <tr><td>Apple</td><td>$1.00</td></tr>
    <tr><td>Banana</td><td>$0.50</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td>$1.50</td></tr>
  </tfoot>
</table>
```

- `<caption>` - provides a title for the table
- `<thead>` - groups header rows
- `<tbody>` - groups body rows
- `<tfoot>` - groups footer rows

Rowspan & Colspan

Cells can span multiple rows or columns.

```html
<table>
  <tr>
    <th colspan="2">Full Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>First</td>
    <td>Last</td>
    <td rowspan="2">25</td>
  </tr>
  <tr>
    <td>Alice</td>
    <td>Smith</td>
  </tr>
</table>
```

Full Name		Age
25	First	Last
	Alice	Smith

- `colspan="2"` - cell spans **2 columns**
- `rowspan="2"` - cell spans **2 rows**
- Borders come from **CSS** (`border` , `border-collapse`) - the old `border="1"` attribute is obsolete

Try It Yourself: Timetable

Build this table (6 minutes), then run it through the W3C Validator:

CET214 Weekly Schedule		
Day	**Time**	**Session**
Sunday	10:00	Lecture
	12:00	Lab
Tuesday	Office hours	

- Use `<caption>`, `<thead>` with `<th scope="col">`, and `<tbody>`
- "Sunday" needs `rowspan="2"`; "Office hours" needs `colspan="2"`

✅ Check: count the cells per row - every row must "fill" 3 columns once spans are included.

Quick Quiz: HTML Elements

Test your understanding! Which tag should you use?

1. You want to display a company logo
2. You want to link to another page
3. You want to group items with bullet points
4. You want to show data in rows and columns
5. You want to make text bold for importance

1. `<img>` (with `alt` text!)
2. `<a href="...">`
3. `<ul>` with `<li>`
4. `<table>`
5. `<strong>` (semantic importance)

Semantic vs Presentational

- `<strong>` indicates **importance**; `<em>` indicates **stress emphasis** that changes meaning.
- `<b>` and `<i>` still exist for text that is styled differently without extra importance (keywords, foreign words, ship names) - for pure looks, use CSS.

T Text Formatting Elements

Beyond `<p>` and headings, HTML provides several inline text elements:

```html
<p>This is <strong>important</strong>.</p>
<p>This is <em>emphasized</em>.</p>
<p>This is <mark>highlighted</mark>.</p>
<p>This is <del>deleted</del>.</p>
<p>This is <ins>inserted</ins>.</p>
<p>This is <sub>subscript</sub>
   and <sup>superscript</sup>.</p>
<p>Use <code>console.log()</code>
   in JavaScript.</p>
```

This is **important**.

This is emphasized.

This is highlighted.

This is ~~deleted~~.

This is inserted.

This is $_{\text{subscript}}$ and ^{superscript}.

Use `console.log()` in JavaScript.

{} HTML Whitespace & Line Breaks

HTML **collapses whitespace** - multiple spaces and newlines become a single space.

- `<br>` - forces a **line break** within a paragraph
- `<hr>` - creates a **horizontal rule** (thematic break)
- `<pre>` - preserves **whitespace and line breaks** exactly as written
- ` ` - inserts a **non-breaking space** (never collapsed, never wrapped: `10 km`)

```html
<p>Roses are red,<br>Violets are blue,<br>HTML is fun,<br>And so are you!</p>
<hr>
<pre>
  This text preserves
    all     spacing
      and line breaks.
</pre>
```

> **Don't space with HTML**
>
> Never use `<br><br>` or rows of ` ` to create spacing - that is CSS's job (`margin`, `gap`).

Putting It All Together

Goal: avoid the classic beginner mistakes and assemble a complete, valid page.

Common Mistakes: HTML Basics

Can you spot what's wrong with each example? Think first, then click to reveal the fix.

1. Missing closing tag

```
<p>Hello World
```

✅ `<p>Hello World</p>`

2. Wrong nesting (overlapping tags)

```
<strong><em>Bold and italic</strong></em>
```

✅ Close in reverse order: `<strong><em>…</em></strong>`

3. Missing `alt` on image

```
<img src="photo.jpg">
```

✅ `<img src="photo.jpg" alt="Our team at the 2025 hackathon">`

4. Missing `href` on link

```
<a>Click here</a>
```

✅ `<a href="schedule.html">Course schedule</a>` - plus descriptive text

Complete HTML5 Page Example

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My Portfolio</title>
    <link rel="icon" href="favicon.svg" type="image/svg+xml">
  </head>
  <body>
    <h1>Welcome to My Portfolio</h1>
    <img src="me.jpg" alt="Portrait of Sara smiling" width="200" height="200">
    <p>I am learning <strong>HTML5</strong> in the
       <em>Web Programming</em> course.</p>
    <h2>My Skills</h2>
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
    <p>Contact me at <a href="mailto:me@example.com">me@example.com</a></p>
  </body>
</html>
```

Summary & Key Takeaways

- HTML describes the **structure and content** of web pages
- Every page needs `<!DOCTYPE html>`, `<html lang>`, `<head>` (with `charset`, `viewport`, `<title>`) and `<body>`
- **Attributes** provide additional information to elements (`href`, `src`, `alt`)
- Use **headings** (`<h1>` – `<h6>`) to create a logical document outline
- **Links** (`<a>`) and **images** (`<img>` with `alt`, `width`, `height`) are fundamental to web content
- **Responsive images**: `srcset` / `sizes`, `<picture>`, and `loading="lazy"` for offscreen images
- HTML provides **three types of lists**: ordered, unordered, and definition
- Use **character entities** for reserved characters (`<`, `&`, `©`)
- Understand the difference between **block** and **inline** elements
- **Tables** organize data into rows and columns with semantic sections

Assignment

1. **Create a personal webpage** using only HTML (no CSS):
   - Include `lang`, `charset`, `viewport`, a page title and a favicon
   - Use headings, paragraphs, and at least one image with `alt`, `width`, `height`
   - Add a responsive image using `srcset` + `sizes` **or** `<picture>`
   - Add a navigation section with links (can be placeholder `#` links)
   - Include an ordered list and an unordered list
2. **Build an HTML table** that displays your weekly class schedule using:
   - `<caption>`, `<thead>`, `<tbody>`
   - At least one use of `colspan` or `rowspan`
3. **Validate** your pages using the W3C HTML Validator
4. **Push** your work to your GitHub fork of the course repository

> **Reminder**
>
> All assignments must be submitted via your **GitHub fork**. Make sure your HTML passes **W3C validation** with no errors!

THANK YOU